

DA3408 Assignment 3

Spark vs. Ray – The Data Engineering Duel

Abhijeet Kumar
DA24B001

For this assignment, I compared Apache Spark and Ray Data on the same NYC Yellow Taxi data-processing task. The main idea was simple: implement the preprocessing pipeline in both frameworks and see how they behave when the data is large enough for distributed processing to actually matter.

The pipeline I used included reading multiple Parquet files, removing null values and duplicates, converting the timestamp columns, filtering invalid trips, joining the trip data with the taxi-zone lookup table, calculating average speed using a Python function, and finally writing the processed data back to Parquet.

I ran both frameworks inside the same Ubuntu ARM64 virtual machine. The VM had 8 GB of RAM and the distributed setup was configured with two CPU workers. Because the machine was relatively small for the size of the data, memory became an important part of the experiment, especially for Ray's global duplicate-removal step.

1 Setup

The dataset consisted of 36 NYC Yellow Taxi Parquet files. Across these files, the input contained

117,790,172

rows.

For Spark, I used Spark 4.2.0 in standalone mode. I configured two executors, with one core and 2 GB of executor memory for each executor.

For Ray, I used Ray 2.59.0 and Ray Data with two CPUs. Both frameworks were run on the same VM so that the comparison was based on the same hardware and input data rather than different machines.

The important thing for me was to keep the actual transformation logic as similar as possible. The Spark pipeline was the reference implementation, and I translated the same stages into Ray Data.

2 Pipeline

The Spark pipeline followed this order:

1. Read all 36 Parquet files.
2. Select the required trip columns.
3. Remove rows containing null values in the required columns.
4. Remove duplicate rows globally.

5. Convert pickup and dropoff values into timestamps.
6. Create the pickup date.
7. Remove invalid trips, such as negative trip distance or dropoff before pickup.
8. Join the trip data with the taxi-zone lookup for pickup and dropoff locations.
9. Calculate average speed using a Python UDF.
10. Write the final result as Parquet.

I followed the same overall structure in Ray. One issue appeared during the implementation: a global duplicate-removal operation creates a large distributed hash shuffle. On my 8 GB VM, this was much more expensive than I initially expected.

3 Spark Result

The complete Spark run finished successfully. The measured total execution time, from the beginning of the pipeline to the Parquet output, was

218.8738 seconds

or roughly 3 minutes and 39 seconds.

The final Spark output contained

78,566,491

rows.

The Python UDF benchmark was run on 100,000 rows and took

0.4463 seconds

.

The Spark run was therefore quite stable on the VM once the local worker and shuffle directories were configured correctly.

4 Ray Result

The completed Ray benchmark took

769.6395 seconds

which is approximately 12.8 minutes.

The completed Ray output contained

116,527,707

rows, and the 100,000-row Python UDF benchmark took

3.2980 seconds

.

This means that, for the completed runs, Spark was approximately

$$\frac{769.6395}{218.8738} \approx 3.52$$

times faster than Ray.

The UDF benchmark also came out lower for Spark in this experiment. The measured Ray UDF time was about 7.39 times the Spark UDF time for the particular benchmark I used.

5 The Duplicate-Removal Problem in Ray

This was the main practical issue I faced during the experiment.

My first Ray implementation performed duplicate removal inside bounded Pandas batches. That approach was much easier on memory and allowed the complete pipeline to finish. However, I realized that this is not exactly equivalent to Spark's global `dropDuplicates()`, because a duplicate can occur in two different batches.

Therefore, I did not treat the Ray output row count as exactly equivalent to the Spark output row count.

I then changed the Ray implementation to perform a real global duplicate removal using Ray's distributed hash aggregation. I also enabled disk-based hash shuffle and increased the number of shuffle partitions in an attempt to reduce memory pressure.

This was the correct direction from a pipeline-parity point of view, but it exposed the hardware limitation very clearly. Ray estimated around 4.0 GiB of memory for the hash aggregation while only about 3.6 GiB was available to that shuffle operation. The process then spent a very long time doing the shuffle and spilling data.

I allowed that run to continue for several hours, but it was not practical to use it as a benchmark. I eventually stopped treating it as a successful run rather than waiting indefinitely for a result. This is also why I am not claiming that the Spark and Ray output row counts match. The completed Ray benchmark and the globally deduplicated Spark benchmark are not directly comparable in output cardinality.

6 UDF Comparison

The assignment also asked for a closer look at the Python UDF.

For 100,000 rows, I measured:

	Spark	Ray
UDF benchmark rows	100,000	100,000
UDF time (seconds)	0.4463	3.2980

One thing I found interesting was that Ray did not automatically win the UDF comparison even though its execution model is Python-oriented. In this particular setup, the Spark measurement was lower.

I would not generalize this result to every Python workload. The UDF is only one small part of the complete pipeline, and the exact implementation, batch size, serialization, and hardware can change the result considerably.

7 Resource and Memory Observations

The VM had only 8 GB of RAM, so memory was a real constraint throughout the experiment. For Spark, I configured the two executors with 2 GB each. Spark completed the full pipeline successfully with this configuration.

For Ray, the global hash shuffle was much more sensitive to available memory. The Ray logs explicitly showed that the estimated memory requirement for the aggregation was around 4.0 GiB, while only around 3.6 GiB was available to the shuffle operation at that point.

I did not capture a reproducible peak CPU and memory time series using `top` or Prometheus during the completed runs. I therefore have not inserted estimated peak percentages into this report. I think reporting a measured number is better than filling the table with a number that I did not actually observe.

8 Which Framework Performed Better?

For this particular workload, my result favors Spark.

The main reason is not just the UDF timing. Most of the workload is structured data processing: global duplicate removal, filtering, joins, timestamp conversion, and Parquet writing. These are operations where Spark performed very well in this environment.

The total measured time was approximately 219 seconds for Spark compared with 770 seconds for the completed Ray implementation.

At the same time, I would not say that Spark is simply “better than Ray” in every situation. Ray is attractive for Python-heavy and AI-oriented workloads, especially when the computation involves more flexible task execution or needs to sit close to other Python machine-learning components.

For a BI-style structured ETL workload like this assignment, however, I would choose Spark based on the measurements I obtained.

9 Performance Tuning Note

I used an LLM during the implementation mainly to translate Spark-style transformations into Ray Data operations and to reason about the memory problem in the Ray global deduplication stage.

One of the suggestions was to use Ray’s disk-based hash shuffle and more shuffle partitions for the global duplicate-removal operation. I applied this change and observed the resulting memory requirement directly from the Ray logs.

The optimization helped move the implementation toward the required global operation, but it did not make the operation practical on the available 8 GB VM. I have therefore not used that incomplete run as the Ray benchmark number.

10 Conclusion

Overall, this experiment gave me a fairly clear result for the hardware and workload I used.

Spark completed the measured pipeline in 218.8738 seconds, while the completed Ray implementation took 769.6395 seconds. Spark was therefore about 3.52 times faster in the completed benchmark.

The biggest difference I noticed was actually during global duplicate removal. Spark handled the operation successfully, whereas Ray’s global hash shuffle became strongly memory-bound on the 8 GB machine.

My final choice for this assignment is therefore **Apache Spark**.

If the same experiment were moved to a larger machine and the workload became more Python/ML-oriented, I would reconsider Ray. But for this structured taxi-data ETL task, the measurements I obtained favor Spark.